Análisis de Brechas y Requerimientos Pendientes - Proyecto DUNAB

Documentación Técnica Generada Automatizada

12 de noviembre de 2026

1. Contexto del Análisis

Este documento contrasta el estado actual del proyecto DUNAB con las especificaciones definidas en el documento *proyectoAnaliticaDatosPDF.pdf*. El propósito es identificar funcionalidades faltantes, incompletas o que requieren mejoras para cumplir con los requerimientos académicos y funcionales establecidos.

La comparación se basa en:

- Las 10 funcionalidades adicionales obligatorias definidas en el PDF.
- Los requerimientos de persistencia, clases modelo y estructura UML.
- Los requerimientos de componentes opcionales y obligatorios.
- Brechas de interfaz de usuario detectadas.

2. Funcionalidades Faltantes Según el PDF

El documento PDF especifica 10 funcionalidades adicionales para el prototipo funcional. A continuación se detalla el estado actual de cada una frente al código implementado.

Funcionalidad 1: Registro e inicio de sesión

Especificación PDF: El estudiante crea una cuenta con sus datos y una contraseña, y luego inicia sesión para ver solo la información de su usuario.

Estado Actual: Implementada. El sistema tiene controlador `Autenticacion.java` con métodos `iniciarSesion()` y `registrarUsuario()`. La vista `LoginFrame.java` permite el ingreso de credenciales y la navegación al dashboard. La persistencia se maneja mediante `UsuarioDAO` con archivos CSV.

Funcionalidad 2: Modo oscuro y modo claro

Especificación PDF: La aplicación permite cambiar entre un tema claro y uno oscuro desde la configuración.

Estado Actual: Implementada. La clase `Tema.java` tiene la variable estática `ModoOscuro` y métodos `aplicarEstiloInput`, `aplicarEstiloBotonPrimario`, etc. que reaccionan a este interruptor. Los botones `botonAlternarTema` en `LoginFrame` y `DashboardFrame` permiten al usuario cambiar de modo. Todos los componentes (tablas, inputs, botones) reaccionan correctamente al cambio.

Funcionalidad 3: Registro de ingresos y gastos

Especificación PDF: El usuario registra ingresos y gastos con su valor, descripción, categoría y fecha.

Estado Actual: Implementada. El controller `FinanzasController.java` tiene el método `registrarTransaccion()` que valida y guarda transacciones. La vista `DashboardFrame.java` (sección Resumen financiero") tiene un formulario con campos para tipo, monto, categoría, descripción y semestre, junto con un botón - Agregar Movimiento". Las transacciones se almacenan en `TransaccionDAO` con archivo CSV.

Funcionalidad 4: Categorías de gastos

Especificación PDF: Los gastos se organizan en categorías como alimentación, transporte, universidad, entretenimiento, compras y otros.

Estado Actual: Parcial. El sistema permite ingresar una categoría libre mediante el campo `campoCategoria` en el formulario de transacciones. Sin embargo, no existe una lista desplegable (JComboBox) con categorías predefinidas obligatorias. El usuario puede escribir cualquier categoría texto. El PDF especifica estas 6 categorías como estándar, pero el código actual no las valía ni las muestra en una lista.

Funcionalidad 5: Barra de búsqueda de movimientos

Especificación PDF: El usuario busca movimientos ya registrados escribiendo palabras de su descripción.

Estado Actual: No implementada. El código actual no incluye un campo de búsqueda o filtro de texto para la tabla de transacciones (`tablaTransacciones`) en la sección Resumen financiero". El modelo de tabla usa `DefaultTableModel` con columnas fijas, pero no hay implementación de filtrado en tiempo real mientras el usuario escribe. Esta es una brecha identificada.

Funcionalidad 6: Presupuesto mensual

Especificación PDF: El estudiante define un límite de gasto para el mes y consulta cuánto ha gastado y cuánto le queda disponible.

Estado Actual: Implementada (parcialmente). El sistema tiene el atributo `presupuestoMensual` en la clase `Usuario` y el campo de edición en la sección "Mi perfil"del dashboard. El controller calcula `calcularSaldoNeto()` que diferencia ingresos y gastos. Sin embargo:

- No existe una validación explícita que impida al usuario exceder el presupuesto definido.
- No hay una visualización dedicada de "quién queda disponible.ªparte del saldo neto general.
- El promedio de gastos mensuales se calcula fijo entre 6 meses (`calcularPromedioMensualGastos`) y no es configurable.

Funcionalidad 7: Metas de ahorro

Especificación PDF: El usuario crea metas con un valor objetivo y registra avances para ver cuánto lleva ahorrado.

Estado Actual: Implementada. El sistema tiene la clase `MetaAhorro.java` con el método `getProgresoPorcentaje()`. El controller `FinanzasController.java` tiene métodos `agregarMetaAhorro()` y `actualizarAvanceMeta()`. La vista `DashboardFrame.java` (sección "Mi progreso") tiene un formulario para crear metas y una tabla (`tablaMetas`) que muestra el progreso porcentual. Los datos se persisten en `MetaAhorroDAO` con archivo CSV.

Funcionalidad 8: Filtros y ordenamiento de movimientos

Especificación PDF: Los movimientos se pueden filtrar y ordenar por fecha, categoría o valor.

Estado Actual: No implementada. El sistema actual no incluye controles de filtrado por fecha, categoría ni valor en la sección de resumen financiero. Las transacciones se muestran en orden de inserción (tabla DefaultTableModel sin ordenamiento dinámico). No hay combos o campos de filtro para refinar la lista de movimientos mostrada.

Funcionalidad 9: Registro y cálculo del costo de materias

Especificación PDF: El estudiante ingresa manualmente el nombre de la materia, el número de créditos y el valor por crédito. La aplicación calcula el costo de cada materia y el total, y estos datos se pueden modificar después.

Estado Actual: Implementada. El sistema tiene la clase `Materia.java` con el método computado `getCostoTotal()`. El controller `FinanzasController.java` tiene métodos `agregarMateria()` y `eliminarMateria()`. La vista `DashboardFrame.java` (sección "Mi progreso") tiene un formulario para inscribir materias con campos de créditos y valor por crédito, y una tabla (`tablaMaterias`) que muestra el costo total. Los datos se persisten en `MateriaDAO`.

Funcionalidad 10: Resumen financiero

Especificación PDF: La pantalla principal muestra un resumen con el saldo, los ingresos, los gastos, el presupuesto, las metas de ahorro y el costo de las materias.

Estado Actual: Implementada. El dashboard (`DashboardFrame.java`) tiene la sección Ïnicioçon tres tarjetas métricas (saldo neto, total gastos, créditos académicos) y un gráfico de barras (`PanelGraficoFinanciero`) que muestra ingresos, gastos y saldo. Además, la sección Resumen financiero"muestra el historial de transacciones y balances. Todos los elementos solicitados están presentes en la interfaz.

3. Requerimiento Adicional de Interfaz: Login - Visibilidad de Contraseña

A continuación se documenta un requerimiento de interfaz pendiente que debe aplicarse en el futuro al formulario de inicio de sesión.

Descripción del Requerimiento

Agregar un ícono/botón con forma de .ºjoübicado en el extremo derecho del campo de contraseña en el formulario de login. Este control debe venir desactivado por defecto (manteniendo la contraseña oculta/enmascarada al escribir). Al hacer clic sobre el ícono del ojo, debe alternar la visibilidad de la contraseña que se está escribiendo en ese momento (mostrar el texto en claro / volver a ocultarlo).

Funcionalidad Detallada

1. **Estado por defecto:** El campo `JPasswordField campoContrasena` en `LoginFrame.java` debe mostrar asteriscos o puntos en lugar del texto legible (comportamiento estándar de JPasswordField, que ya está activo por defecto).

2. **Control .ºjo":** Un botón o ícono circular ubicado a la derecha del campo de contraseña, alineado verticalmente en el centro.

3. **Comportamiento inicial:** El botón comienza en estado çerrado"(ojo tachado o ícono de ojo cerrado), y el campo de contraseña mantiene el enmascarado (mostrando asteriscos).

4. **Al hacer clic:** Al hacer clic en el ícono/botón, se debe:

   - Cambiar el ícono a .ªbierto"(ojo sin tachar).
   - Cambiar el campo `campoContrasena` de `JPasswordField` a un campo de texto plano (`JTextField`) o alternar el modo de máscara usando `JPasswordField` con la propiedad de máscara desactivada temporalmente.
   - Mostrar el texto de la contraseña en claro mientras el usuario escribe.

5. **Al hacer clic nuevamente:** El comportamiento se invierte: el ícono vuelve a çerrado", y el campo vuelve a enmascarar el texto (mostrando asteriscos nuevamente).

Archivos del Proyecto Afectados

El cambio debe aplicarse en el siguiente archivo:

`src/com/unab/dunab/view/LoginFrame.java`

Específicamente, se debe modificar el método `crearInterfaz()` o añadir un nuevo método que:

- Cree un botón `JButton` con ícono de ojo (se pueden usar íconos de texto o imágenes/iconos registrados para abierto y cerrado).
- Posicione el botón en la celda de la contraseña usando el gestor de layout GridBagLayout (consistente con el resto del formulario).
- Agregue un `ActionListener` que alterne entre los modos de visibilidad.
- Tenga como estado inicial **desactivado/oculto** por defecto (la contraseña debe venir enmascarada por defecto, que es el comportamiento estándar de JPasswordField).

Notas de Implementación

- **No se implementa en este momento** en el código actual, pero se documenta para desarrollo futuro.
- El campo `campoContrasena` actualmente es un `JPasswordField` en línea 139 de LoginFrame.java, que ya enmascara la entrada por defecto.
- Se debe tener cuidado de no afectar el método `iniciarSesion()` que lee el contenido con `new String(campoContrasena.getPassword())`.
- Se pueden reutilizar los patrones de estilo ya definidos en la clase `Tema.java` para botones y inputs.

4. Brechas Adicionales Detectadas

Más allá de las 10 funcionalidades del PDF, se identificaron las siguientes brechas:

Brecha	Descripción

Búsqueda de transacciones	No existe funcionalidad de búsqueda por descripción en la tabla de movimientos. El usuario no puede encontrar un movimiento específico sin revisar toda la lista.
Filtro por categoría	No hay filtro para mostrar solo transacciones de una categoría específica (ej. solo .ªlimentación").
Ordenamiento por fecha	La tabla de transacciones no permite ordenar clicando en las columnas de fecha. El sistema siempre muestra en orden de inserción.
Validación de presupuesto	No existe validación que compare los gastos registrados contra el presupuesto mensual del usuario, ni alertas cuando se acerca el límite.
Componentes opcionales faltantes	El PDF menciona componentes opcionales que no han sido desarrollados, como validaciones avanzadas de formularios, mensajes de error más amigables, y ayuda contextual.
Rankings con datos dinámicos	Los rankings actuales (sección Rankings"del dashboard) muestran datos fijos de ejemplo. No hay integración con cálculos reales de progreso de los usuarios registrados.
Exportación con datos de muestra	Al generar reportes, si el usuario no tiene datos, el PDF/Excel vacío no incluye mensajes informativos, solo estructura vacía.

5. Conclusión

El proyecto DUNAB cumple con el 70 % de las funcionalidades obligatorias definidas en el documento PDF. Las funcionalidades mejor implementadas son: autenticación, registro de transacciones, cálculo de costos de materias y modos de tema. Las principales brechas están en: búsqueda y filtrado de movimientos, validación de presupuesto interactiva, y componentes de interfaz adicionales como el ícono de visibilidad de contraseña en el login.

Los próximos pasos de desarrollo deberían priorizar:

1. Implementar el buscador/filtro de transacciones por descripción y categoría.
2. Agregar la funcionalidad de visibilidad de contraseña en el login (como requerimiento adicional documentado arriba).
3. Mejorar la visualización del avance de presupuesto mensual con indicadores visuales.
4. Cargar datos dinámicos en la sección de Rankings basado en el progreso real de los usuarios.